
ENGENHARIA DE SOFTWARE

2.º Ano · Ano Letivo 2025/2026

RELATÓRIO DE PROJETO DE SOFTWARE

the 100's

Sistema de Gestão de Loja de Vinhos Premium

"Bottled Memories — uma curadoria dos melhores vinhos portugueses, do Douro ao Alentejo, do Porto à Madeira."

REALIZADO POR

Eduardo Saavedra Lourenço

Kollan Andre Gafuro Intacua

Pedro Gabriel Matias Gomes

Resumo

O presente documento constitui o **Relatório de Projeto de Software** do projeto *the 100's — Sistema de Gestão de Loja de Vinhos Premium*, uma aplicação web desenvolvida no âmbito da unidade curricular de Engenharia de Software do 2.º ano da Licenciatura em Engenharia Informática da Universidade Lusófona do Porto.

O *the 100's* é uma solução integrada e auditável para a gestão de uma loja de vinhos *premium*, cobrindo ponto de venda (POS), inventário, gestão visual de caves, devoluções, *dashboard* executivo e relatórios analíticos com *exports* para Excel. O sistema foi desenvolvido numa arquitetura de três camadas — *frontend* estático em HTML, CSS e JavaScript Vanilla, *backend* em Python/Flask com API REST e base de dados MySQL 8 (com *fallback* automático para SQLite) —, com autenticação baseada em *tokens Bearer* e controlo de acessos por papéis.

Este relatório documenta todas as dimensões do projeto: âmbito e decomposição funcional, gestão de dados e modelo relacional, estratégia de testes, plano de formação, segurança, gestão de risco, plano de manutenção e conclusões.

Palavras-chave

Gestão de Projeto, API REST, Flask, Python, MySQL, JavaScript *Vanilla*, POS, Loja de Vinhos, RGPD, *Bottled Memories*.

Repositório do Projeto

Código-fonte disponível em: github.com/PedroGGomess/Vinha-D-Ouro

Índice

Resumo	i
1 Introdução	1
1.1 Enquadramento Inicial	1
1.2 Definição e Objetivos do Documento	1
1.3 Público-Alvo	1
1.4 Breve Resumo do Documento	2
1.5 Propósito do Projeto	2
1.6 Repositório do Código-Fonte	2
2 Âmbito do Projeto	3
2.1 Apresentação do Projeto	3
2.2 Descrição da Área de Negócio	3
2.3 Problemas no Negócio Antes do Sistema	3
2.4 Como o Sistema Resolve os Problemas	3
2.5 Decomposição Funcional do Sistema	4
2.6 Perfis de Utilizador e Acessos	4
2.7 Modelagem do Comportamento (UML e Processos)	5
2.7.1 Diagramas de Casos de Uso (UML)	5
2.7.2 Fluxogramas de Processo	6
2.8 Interface Gráfica (Frontend)	6
2.9 Desempenho e Disponibilidade Esperadas	8
2.10 Constrangimentos do Processo	9
3 Documentação	10
3.1 Enumeração dos Documentos Produzidos	10
3.2 Descrição de Cada Documento	10
3.2.1 Plano de Projeto de Software (D1)	10
3.2.2 Especificação de Requisitos (D2)	10

3.2.3	Modelo de Dados (D3)	11
3.2.4	Documentação da API REST (D4)	11
3.2.5	Plano de Testes (D5)	11
3.2.6	Guias de Instalação (D6)	11
3.2.7	Relatório de Projeto de Software (D7)	11
4	Gestão de Dados	12
4.1	Arquitetura de Software em Camadas	12
4.2	Apresentação da Base de Dados	13
4.3	Permissões de Acesso a Dados	14
4.4	Medidas de Conformidade com o RGPD	14
4.5	Estado Inicial da Base de Dados	15
4.6	Tarefas de Carregamento Inicial de Dados	15
4.7	Ambiente de Execução e Instalação Física	15
5	Realização de Testes	16
5.1	Objetivos do Plano de Testes	16
5.2	Descrição dos Níveis de Integridade	16
5.3	Diagramas de Sequência de Integração	16
5.4	Tipos de Testes Realizados	17
5.4.1	Testes Unitários	18
5.4.2	Testes de Integração da API	18
5.4.3	Testes E2E (Fluxos Completos)	18
5.5	Geração de Dados para Testes	18
5.6	Critérios de Aceitação por Fase	19
6	Formação	20
6.1	Métodos de Formação	20
6.2	Recursos para Cada Método	20
6.3	Bases de Dados da Formação	20
6.4	Duração da Formação	20
6.5	Participantes, Responsabilidades e Avaliação	21
7	Segurança	22

7.1	Segurança no Desenvolvimento	22
7.2	Testes de Segurança	22
7.3	Segurança no Alojamento da Base de Dados	23
7.3.1	Segurança contra Acessos Não Autorizados (<i>Security</i>)	23
7.3.2	Integridade dos Dados (<i>Safety</i>)	23
7.4	Realização de Backups	23
8	Gestão de Risco	24
8.1	Impacto do Risco	24
8.2	Identificação e Análise dos Riscos	24
8.3	Plano de Resposta aos Riscos Identificados	25
9	Manutenção	26
9.1	Modelo de Manutenção Adotado	26
9.2	Apoio ao Utilizador Após a Entrega	26
9.3	Periodicidade da Manutenção	26
9.4	Ações a Realizar na Manutenção	26
9.5	Modo de Realizar a Manutenção	27
9.6	Responsáveis pela Manutenção	27
10	Conclusões	28
10.1	Reflexões Finais sobre a Condução do Projeto	28
10.2	Áreas a Melhorar	28
10.3	Perspetivas de Trabalho Futuro	29
	Referências	30

Lista de Tabelas

2.1	Matriz de acessos por perfil de utilizador do sistema.	5
3.1	Documentos produzidos no âmbito do projeto <i>the 100's</i>	10
5.1	Critérios de aceitação por fase de desenvolvimento.	19
6.1	Recursos necessários para cada método de formação.	20
8.1	Matriz de identificação e análise de riscos do projeto.	24
8.2	Estratégias de mitigação e resposta aos riscos identificados.	25
9.1	Periodicidade da manutenção por tipo.	26

Lista de Figuras

2.1	Diagrama UML do Caso de Uso UC01: Realizar Venda no POS.	5
2.2	Fluxograma de tomada de decisão: Processo de Venda no POS.	6
2.3	Interface gráfica do utilizador: <i>Dashboard</i> executivo com KPIs e alertas.	7
2.4	Interface gráfica do utilizador: Ponto de venda com catálogo, carrinho e <i>checkout</i>	7
2.5	Interface gráfica do utilizador: Gestão visual das caves A, B e C com <i>slots</i>	8
2.6	Interface gráfica do utilizador: Relatórios analíticos e <i>export</i> para Excel.	8
4.1	Diagrama da arquitetura de <i>software</i> lógica em 3 camadas do <i>the 100's</i>	12
4.2	Diagrama Entidade-Relacionamento (ER) completo da base de dados vinhadouro.	14
4.3	Diagrama de arquitetura física e <i>deployment</i> em ambiente <code>localhost</code>	15
5.1	Diagrama de Sequência DS01: Processo de <i>login</i> e emissão de <i>token</i>	17
5.2	Diagrama de Sequência DS02: Processo de <i>checkout</i> com débito atómico de <i>stock</i>	17

1.1 Enquadramento Inicial

O desenvolvimento de *software* envolve decisões técnicas, organização do trabalho, gestão de riscos e validação contínua do progresso. Mesmo num contexto académico, a ausência de planeamento tende a gerar retrabalho, inconsistências entre módulos e dificuldade em demonstrar que o sistema cumpre os requisitos definidos.

O *the 100's* surge como resposta às necessidades típicas de uma loja de vinhos *premium*: catálogo extenso, vendas com múltiplos métodos de pagamento, gestão de *stock* sensível, controlo físico de caves com condições específicas de armazenamento e necessidade de manter rasto digital de cada operação para fins de auditoria. O sistema pretende substituir práticas informais — papel, folhas de cálculo dispersas e aplicações isoladas — por uma solução centralizada, segura e rastreável.

1.2 Definição e Objetivos do Documento

Este relatório documenta o ciclo completo de desenvolvimento do *the 100's*, cobrindo os aspetos técnicos, organizacionais e de qualidade do projeto. Os principais objetivos são:

- Documentar o âmbito, a arquitetura e as funcionalidades do sistema;
- Descrever as decisões técnicas tomadas durante o desenvolvimento;
- Apresentar a estratégia de testes e os resultados obtidos;
- Definir os planos de segurança, formação e manutenção;
- Identificar riscos e as respetivas estratégias de mitigação;
- Refletir sobre o trabalho realizado e perspetivas futuras.

1.3 Público-Alvo

O documento destina-se a:

- **Equipa de desenvolvimento** (3 elementos), como registo de decisões e base de conhecimento;
- **Docente / avaliadores**, como base de verificação de metodologia e consistência;
- **Stakeholders simulados** (Gerente, Funcionário de loja, Armazenista, Administrador), como referência de objetivos funcionais e regras de negócio.

1.4 Breve Resumo do Documento

O relatório está estruturado em dez capítulos principais, seguindo a estrutura definida para o Relatório de Projeto de Software da unidade curricular. Começa pelo âmbito e decomposição funcional do sistema, avança para a documentação produzida, gestão de dados, testes, formação, segurança, gestão de risco e manutenção, concluindo com reflexões finais e perspetivas de trabalho futuro.

1.5 Propósito do Projeto

O *the 100's* é uma aplicação web completa para a gestão integral de uma loja de vinhos *premium*, suportando:

- Registo e manutenção de catálogo (vinhos por tipo, região, produtor, preço, *stock*);
- Ponto de venda com quatro métodos de pagamento e emissão de fatura digital com QR code;
- Gestão de caves físicas (A, B, C) com mapeamento visual de *slots*;
- Devoluções com reposição automática de *stock* e auditoria;
- *Dashboard* executivo editável com KPIs em tempo real;
- Relatórios analíticos com *export* para Excel.

O propósito operacional é reduzir erros manuais, aumentar a rastreabilidade e a transparência das operações, garantir o controlo do *stock* e melhorar a tomada de decisão pela gestão.

1.6 Repositório do Código-Fonte

Todo o código-fonte do projeto, bem como os *scripts* de inicialização da base de dados, ficheiros de configuração e documentação técnica, estão disponíveis no repositório público do GitHub:

GitHub

<https://github.com/PedroGGomess/Vinha-D-Ouro>

2.1 Apresentação do Projeto

O *the 100's* é um sistema de informação web desenvolvido para digitalizar e centralizar os processos operacionais de uma loja de vinhos *premium*. O sistema suporta o ciclo completo: *cliente / catálogo* → *carrinho* → *checkout* → *fatura digital* → *stock atualizado* → *análise de vendas*.

A identidade visual — *Bottled Memories* — combina uma paleta *gold + ink* com tipografia *Cormorant Garamond* e *Inter*, conferindo ao produto uma personalidade *premium* adequada ao segmento.

2.2 Descrição da Área de Negócio

O negócio do retalho especializado de vinhos *premium* caracteriza-se por elevada variação de preço unitário, exigências específicas de armazenamento (temperatura, humidade) e por uma relação próxima com clientes habituais cujas preferências importa registar. A operação diária envolve três funções distintas:

- **Loja:** Atendimento, venda, emissão de fatura, foco no cliente.
- **Armazém:** Gestão de *stock*, reposição em caves, foco na disponibilidade.
- **Gestão:** Visão consolidada do desempenho, rentabilidade e controlo.

2.3 Problemas no Negócio Antes do Sistema

Antes de um sistema integrado, surgem problemas recorrentes:

1. **Inventário em folhas de cálculo:** dessincronização entre o *stock* real e o registado, com risco de venda de produtos sem disponibilidade.
2. **Faturação manual:** emissão lenta, propensa a erros e sem rasto digital pesquisável.
3. **Ausência de visão consolidada:** gestão sem KPIs em tempo real.
4. **Gestão de caves sem mapeamento:** dificuldade em localizar fisicamente cada garrafa.
5. **Devoluções sem controlo:** reposição manual de *stock*, com falhas de auditoria.
6. **Acesso indiferenciado:** todos os colaboradores partilhavam as mesmas credenciais.

2.4 Como o Sistema Resolve os Problemas

O *the 100's* responde a estes problemas através de:

- **Centralização de dados:** toda a informação numa única base de dados acessível pelos quatro papéis de utilizador.
- **POS completo:** catálogo agrupado por tipo, carrinho, quatro métodos de pagamento (Cartão, Numerário com cálculo de troco, MB WAY e Multibanco com referência) e atualização atómica de *stock* por venda.
- **Fatura digital com QR code:** gerada via `api.qrserver.com` e enviada por Email, SMS ou WhatsApp.
- **Cave 3D:** gestão visual de *slots* em 3 caves (A/B/C) com persistência.
- **Devoluções com auditoria:** pesquisa em tempo real, reposição automática de *stock* e registo do movimento.
- **Autenticação por tokens:** validade de 8 horas, quatro papéis (FUNCIONARIO, GERENTE, ARMAZENISTA, ADMIN) e proteção por *endpoint*.
- **Dashboard de KPIs:** painel editável com vendas do dia, *top* vinhos, alertas de *stock* e estado das operações em tempo real.

2.5 Decomposição Funcional do Sistema

O sistema é decomposto em sete módulos principais:

1. **Autenticação & Sessão** — *Login*, gestão de *token Bearer* e *logout*.
2. **Catálogo & POS** — CRUD de vinhos, carrinho, *checkout* e emissão de fatura.
3. **Inventário (Stock)** — CRUD, alertas de *stock* mínimo, auditoria de movimentos.
4. **Caves** — Mapeamento visual A/B/C, alocação de garrafas a *slots*.
5. **Vendas & Devoluções** — Histórico de vendas, detalhe, processamento de devoluções.
6. **Dashboard & Relatórios** — KPIs agregados, análises mensais, exports Excel.
7. **Equipa & Utilizadores** — Gestão de contas e papéis.

2.6 Perfis de Utilizador e Acessos

O sistema implementa quatro perfis distintos com segregação de funções:

Módulo / Página	ADMIN	Gerente	Funcionário	Armazenista
Catálogo (POS)	Total	Total	Total	Leitura
Inventário (Stock)	Total	Leitura	Negado	Total
Caves	Total	Leitura	Negado	Total
Dashboard & KPIs	Total	Total	Negado	Negado
Vendas e Devoluções	Total	Total	Próprias	Negado
Relatórios / Excel	Total	Total	Negado	Negado
Gestão de Equipa	Total	Leitura	Negado	Negado

Tabela 2.1. Matriz de acessos por perfil de utilizador do sistema.

2.7 Modelagem do Comportamento (UML e Processos)

Para validar e estruturar a reatividade do sistema face às regras impostas, foram desenvolvidos diagramas de Casos de Uso (UML) e fluxogramas operacionais.

2.7.1 Diagramas de Casos de Uso (UML)

Os diagramas seguintes descrevem a interação dos atores com o limite do sistema do *the 100's*, evidenciando as dependências obrigatórias (*include*) e os desvios de exceção (*extend*).

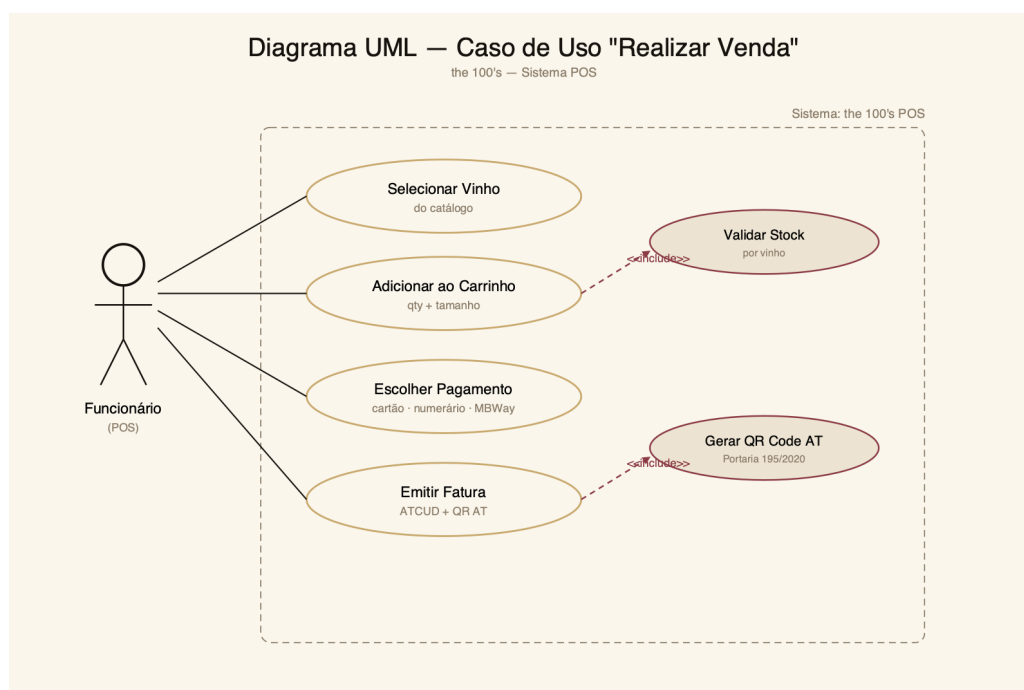


Figura 2.1. Diagrama UML do Caso de Uso UC01: Realizar Venda no POS.

2.7.2 Fluxogramas de Processo

Os fluxogramas detalham a árvore de decisão executada nas camadas aplicacionais durante as operações de venda e gestão de *stock*.

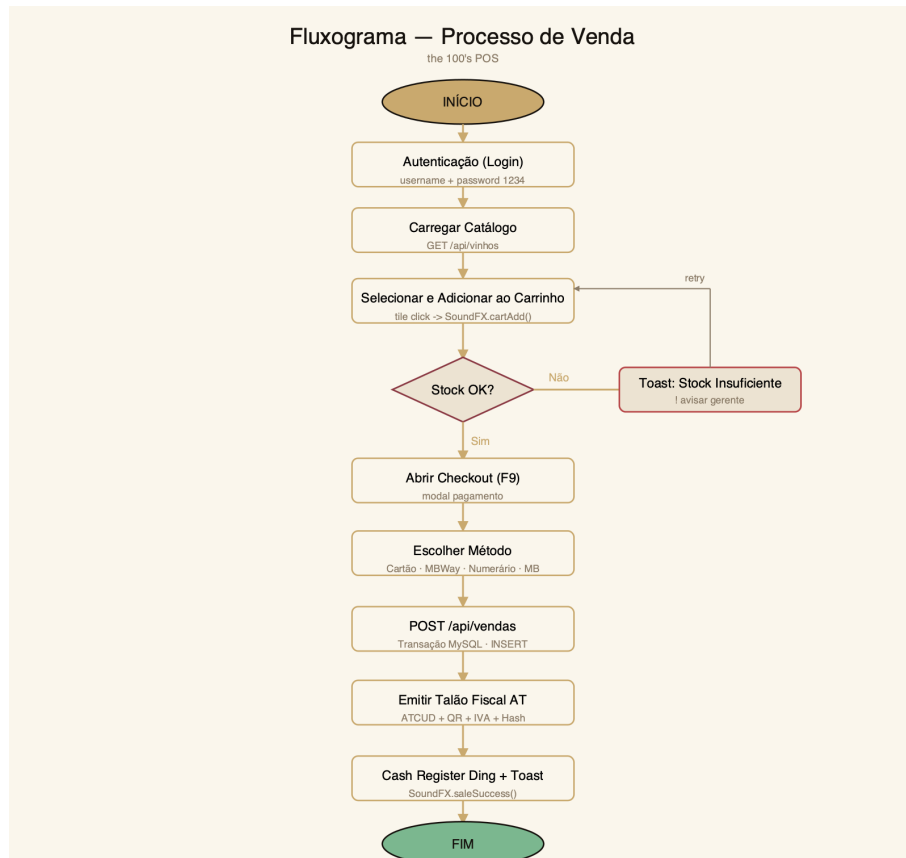


Figura 2.2. Fluxograma de tomada de decisão: Processo de Venda no POS.

2.8 Interface Gráfica (Frontend)

A camada visual interage diretamente com o utilizador, renderizando os dados da API em tempo real com a identidade *Bottled Memories*.

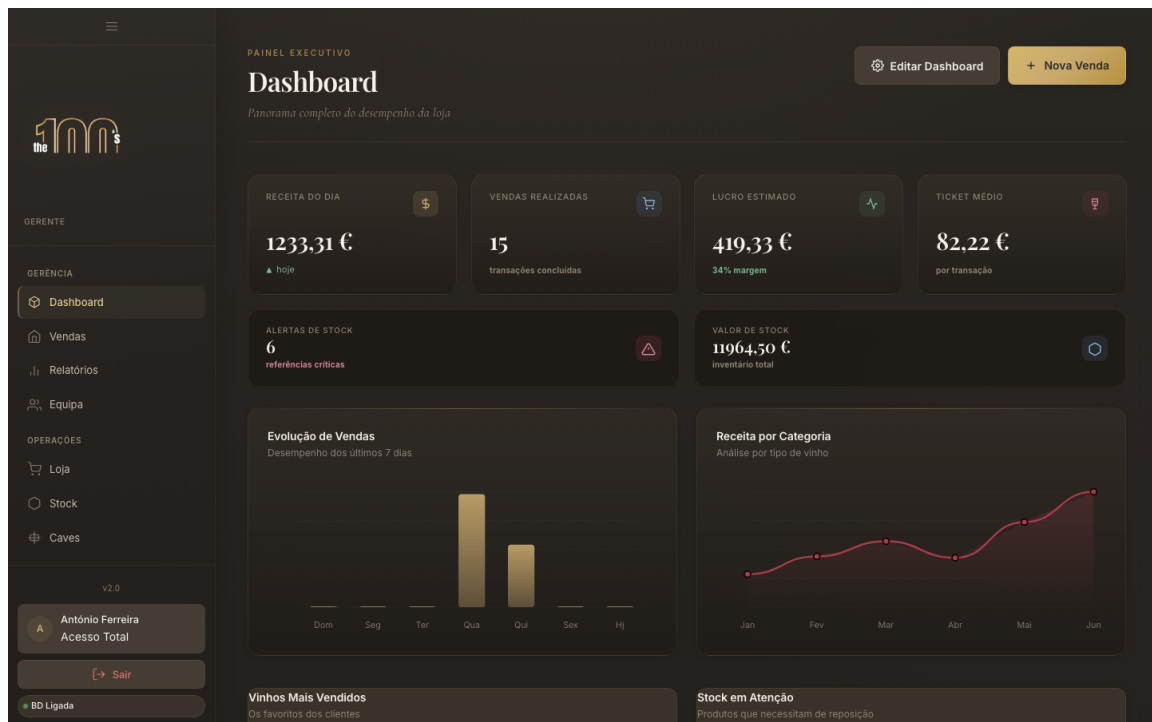


Figura 2.3. Interface gráfica do utilizador: *Dashboard* executivo com KPIs e alertas.

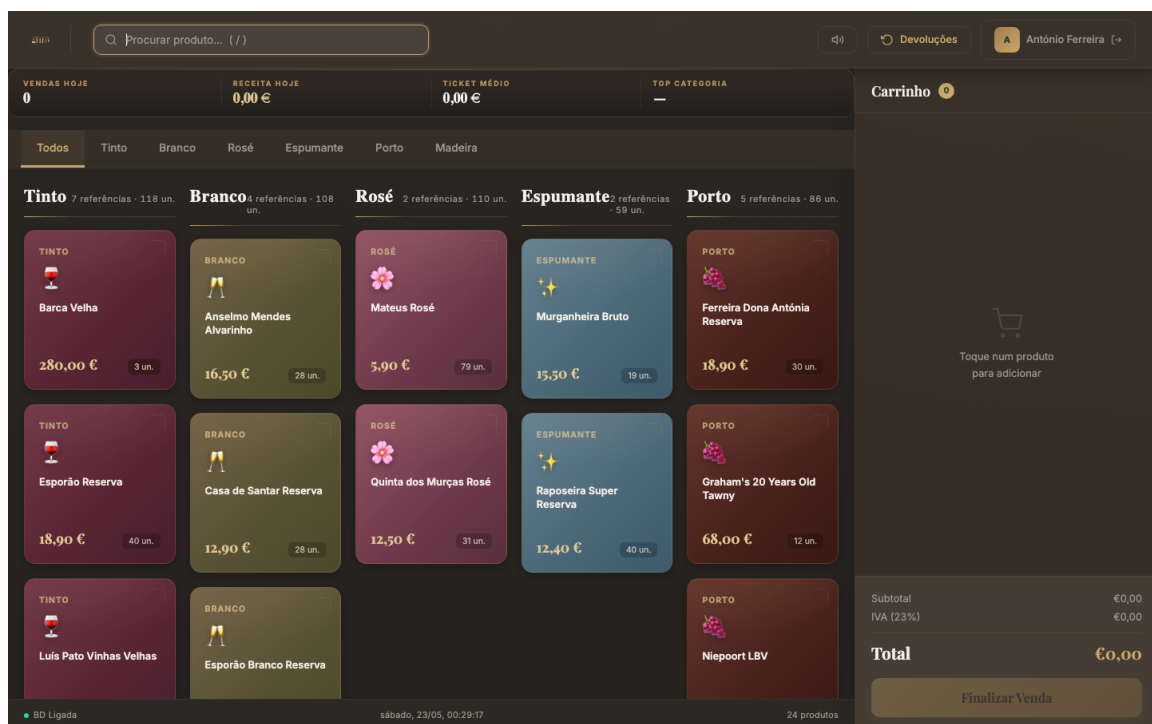


Figura 2.4. Interface gráfica do utilizador: Ponto de venda com catálogo, carrinho e *checkout*.

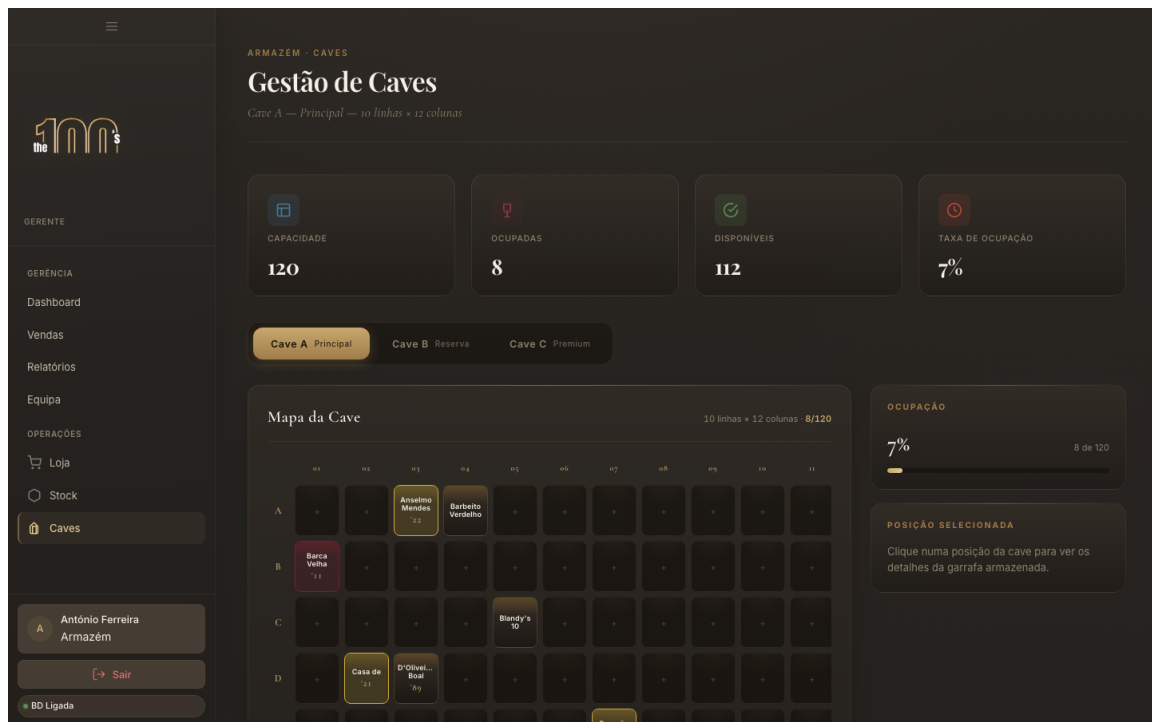


Figura 2.5. Interface gráfica do utilizador: Gestão visual das caves A, B e C com slots.



Figura 2.6. Interface gráfica do utilizador: Relatórios analíticos e export para Excel.

2.9 Desempenho e Disponibilidade Esperadas

Para garantir a fluidez na operação diária da loja, o *the 100's* deverá cumprir as seguintes métricas de serviço:

Métricas de Serviço

- **Disponibilidade (*Uptime*):** 99% em horário de operação da loja.
- **Tempo de Resposta:** Listagens e pesquisas em menos de 200 ms; *checkout* em menos de 1 segundo.
- **Concorrência:** Suporte até 4 utilizadores simultâneos sem degradação perceptível.
- **Tolerância a falhas:** *Fallback* automático para SQLite mantém o catálogo e o POS operacionais perante falhas do MySQL.

2.10 Constrangimentos do Processo

As principais regras de negócio e limites do âmbito são:

- *Stock* validado a nível atômico — uma venda só é aceite se o *stock* disponível for igual ou superior à quantidade pedida.
- Toda a venda gera obrigatoriamente um código único no formato VD-AAAA-NNN.
- As *passwords* são guardadas exclusivamente em *hash* (`pbkdf2:sha256` ou `scrypt`); *plaintext* apenas no *seed* inicial, com *auto-upgrade* no primeiro *login*.
- O sistema está desenhado para correr em `localhost` — sem dependência de *cloud*.

Ficam **fora do âmbito** desta versão: certificação fiscal AT, integrações externas com terminais de pagamento ou ERP, e arquitetura multi-loja.

3 Documentação

3.1 Enumeração dos Documentos Produzidos

No âmbito deste projeto foram produzidos os seguintes documentos, todos versionados em docs/ no repositório Git:

N.º	Documento	Redator	Aprovado por
D1	Plano de Projeto de Software	Equipa (todos)	Pedro Strecht
D2	Especificação de Requisitos (RF/RNF)	Pedro G.	Equipa
D3	Modelo de Dados (ER + <i>script</i> SQL)	Eduardo L.	Grupo 13
D4	Documentação da API REST (25 <i>end-points</i>)	Eduardo L.	Equipa
D5	Plano de Testes	Pedro G.	Equipa
D6	Guias de Instalação (Mac/Linux + Windows)	Kollan I.	Equipa
D7	Relatório de Projeto de Software (este documento)	Pedro G.	Pedro Strecht

Tabela 3.1. Documentos produzidos no âmbito do projeto *the 100's*.

3.2 Descrição de Cada Documento

3.2.1 Plano de Projeto de Software (D1)

Documento orientador do ciclo de desenvolvimento, com calendário em *sprints* de duas semanas. Define âmbito, responsabilidades, *standards* de codificação (PEP-8 no *backend*, BEM no CSS) e gestão de riscos.

3.2.2 Especificação de Requisitos (D2)

Levantamento sistemático dos requisitos funcionais e não funcionais. Identifica atores, casos de uso principais (*login*, venda, devolução, gestão de *stock*, relatórios) e qualidades exigidas (segurança, desempenho, usabilidade).

3.2.3 Modelo de Dados (D3)

Diagrama Entidade-Relacionamento (ER) da base de dados vinhadouro, incluindo o *script* `setup_mysql.sql` de criação do esquema (9 tabelas, 4 vistas, 13 índices) e os dados de *seed* (22 vinhos, 8 vendas históricas, 12 movimentos de *stock*).

3.2.4 Documentação da API REST (D4)

Tabela com os 25 *endpoints* REST do sistema (método HTTP, rota, *roles* autorizados e descrição). Todas as escritas estão protegidas por decoradores Python `@requires_auth` ou `@requires_role(...)`.

3.2.5 Plano de Testes (D5)

Documento com objetivos, níveis de integridade, tipos de teste (unitário, integração, sistema, aceitação), cenários e critérios de aceitação por fase.

3.2.6 Guias de Instalação (D6)

Dois ficheiros Markdown — `COMO_CORRER.md` para macOS/Linux e `WINDOWS.md` para Windows — com instruções passo-a-passo, *troubleshooting* (porta 8080 ocupada, *password* do MySQL, etc.) e o procedimento de *fallback* para SQLite.

3.2.7 Relatório de Projeto de Software (D7)

O presente documento, que sumariza e documenta todo o projeto conforme a estrutura definida pelo docente.

4.1 Arquitetura de Software em Camadas

A infraestrutura aplicacional do *the 100's* baseia-se no padrão arquitetural de três camadas, isolando as responsabilidades de UI, processamento e persistência relacional. Os dados são introduzidos pelos utilizadores através da interface web e enviados para o *backend* via pedidos HTTP/JSON. O *backend* Flask valida e persiste os dados numa base de dados relacional MySQL 8, acedida via PyMySQL com `DictCursor`, com *fallback* automático para SQLite. Não existe transferência de dados para sistemas externos.



Figura 4.1. Diagrama da arquitetura de *software* lógica em 3 camadas do *the 100's*.

A camada intermédia de *Services* no Flask valida as transações antes de invocar a camada de persistência. O excerto seguinte ilustra o encapsulamento da regra de negócio de débito atómico de *stock* numa venda:

```

1 @app.route("/api/vendas", methods=["POST"])
2 @requires_auth
3 def criar_venda():
4     payload = request.get_json()
5     itens = payload.get("itens", [])
6
7     with db_transaction() as cur:
8         for item in itens:
9             cur.execute(

```

```
10         "SELECT quantidade FROM vinhos WHERE id = %
11             s FOR UPDATE",
12         (item["vinho_id"],),
13     )
14     stock_atual = cur.fetchone()["quantidade"]
15     if stock_atual < item["quantidade"]:
16         raise StockInsuficienteError(
17             f"Vinho {item['vinho_id']}: stock = {
18                 stock_atual}, "
19             f"pedido = {item['quantidade']}"
20         )
21     cur.execute(
22         "UPDATE vinhos SET quantidade = quantidade
23         - %s WHERE id = %s",
24         (item["quantidade"], item["vinho_id"]),
25     )
26     # ... persistir cabeçalho de venda + itens_venda +
27     # movimento de stock
28     return jsonify({"codigo": novo_codigo}), 201
```

Listing 4.1. Validação atômica de stock no *checkout* (excerto de `server.py`).

4.2 Apresentação da Base de Dados

A base de dados vinhadouro usa o *charset* `utf8mb4` com *collation* `utf8mb4_unicode_ci`. Está composta pelas seguintes entidades principais:

- `peessoas` — Base de pessoas (clientes e funcionários).
- `clientes` — Especialização de pessoa para cliente (NIF, preferências em JSON).
- `funcionarios` — Dados laborais (cargo, salário, nível).
- `utilizadores` — Credenciais (`username`, `password_hash`, `role`).
- `vinhos` — Catálogo (22 referências *seed*; preço, quantidade, *stock* mínimo).
- `caves` — Locais de armazenamento (3 caves; temperatura e humidade ideais).
- `vendas` — Cabeçalho de venda (código único, método de pagamento, estado).
- `itens_venda` — Linhas de cada venda (quantidade, preço unitário, subtotal).
- `movimentos_stock` — Auditoria de entradas/saídas (tipo, motivo, funcionário).

Foram ainda definidas 4 vistas SQL (`v_stock_baixo`, `v_vendas_hoje`, `v_top_vinhos`, `v_movimentos_resumo`) e 13 índices secundários que mantêm tempos de resposta abaixo dos 200 ms.

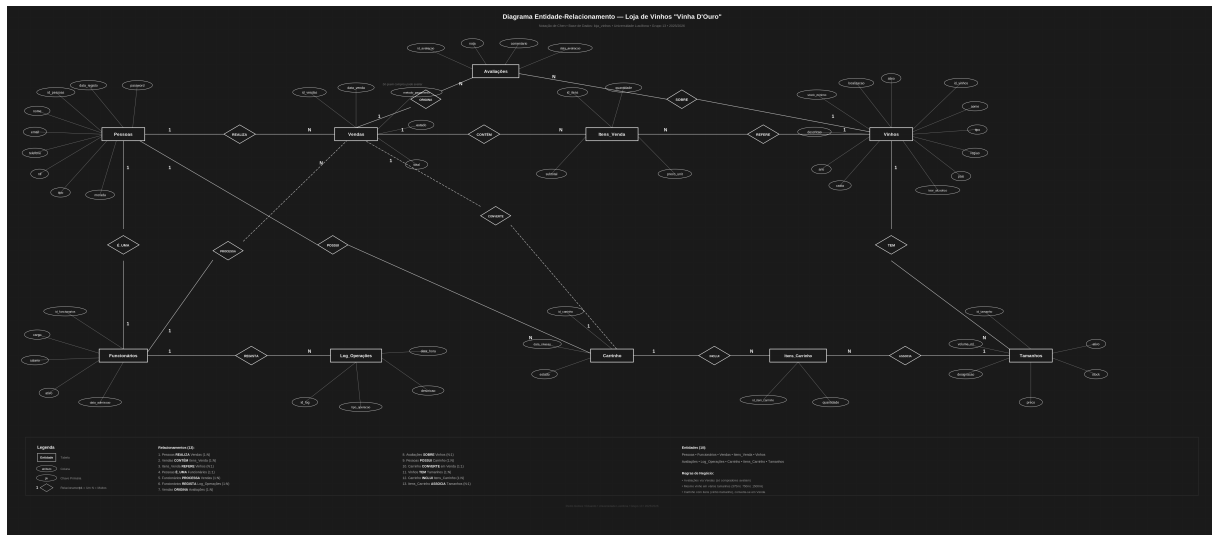


Figura 4.2. Diagrama Entidade-Relacionamento (ER) completo da base de dados vinhadouro.

4.3 Permissões de Acesso a Dados

O acesso aos dados é controlado a dois níveis:

- **Nível de API (RBAC):** cada *endpoint* REST é protegido com decoradores Python (`@requires_auth` para *token* válido e `@requires_role(...)` para um papel específico), antes de executar qualquer operação.
- **Nível de base de dados:** a aplicação liga ao MySQL com um utilizador dedicado (`sgo_user`) com permissões limitadas ao esquema `vinhadouro` (`SELECT`, `INSERT`, `UPDATE`, `DELETE`). Não são concedidos privilégios de DDL em produção.

A matriz completa de permissões está apresentada na Tabela 2.1 (capítulo 2).

4.4 Medidas de Conformidade com o RGPD

Dado que o sistema armazena dados pessoais de clientes (nome, NIF, telefone, morada), foram adotadas as seguintes medidas de conformidade com o Regulamento Geral sobre a Proteção de Dados (RGPD):

- **Minimização de dados:** apenas se recolhem os dados estritamente necessários.
- **Passwords cifradas:** *hash pbkdf2:sha256* ou *script* via `werkzeug.security`, nunca em texto claro.
- **Controlo de acessos:** apenas GERENTE e ADMIN consultam a lista completa de clientes.
- **Direito ao esquecimento:** *soft delete* via campo `ativo`, preservando integridade referencial mas removendo o cliente da operação.
- **Rastreabilidade:** todas as escritas registam o utilizador autor e o *timestamp*.

4.5 Estado Inicial da Base de Dados

A base de dados é criada a partir do *script* `setup_mysql.sql`, que define o esquema relacional completo com restrições de integridade (*foreign keys*, *unique constraints*). O *script* é idempotente: utiliza `INSERT IGNORE` e `CREATE TABLE IF NOT EXISTS`, podendo ser executado várias vezes sem destruir dados existentes.

4.6 Tarefas de Carregamento Inicial de Dados

Para fins de desenvolvimento e demonstração, o *script* povoa a base de dados com:

- 8 pessoas (4 funcionários, 4 clientes, 1 cliente balcão).
- 4 utilizadores (gerente, loja, stock, admin) com *password* inicial 1234 — atualizada para *hash* no primeiro *login*.
- 3 caves configuradas (Principal, Secundária, Montra Loja).
- 22 vinhos *seed* distribuídos por Tinto, Branco, Rosé, Espumante, Porto e Madeira.
- 8 vendas históricas (últimos 7 dias) com 20 linhas de itens.
- 12 movimentos de *stock* (entradas iniciais e saídas correspondentes às vendas).

4.7 Ambiente de Execução e Instalação Física

O *deployment* corre localmente na máquina do estabelecimento. O servidor Flask responde na porta 8080, o MySQL na porta 3306 (apenas `localhost`), e o utilizador acede via *browser* a `http://localhost:8080`.

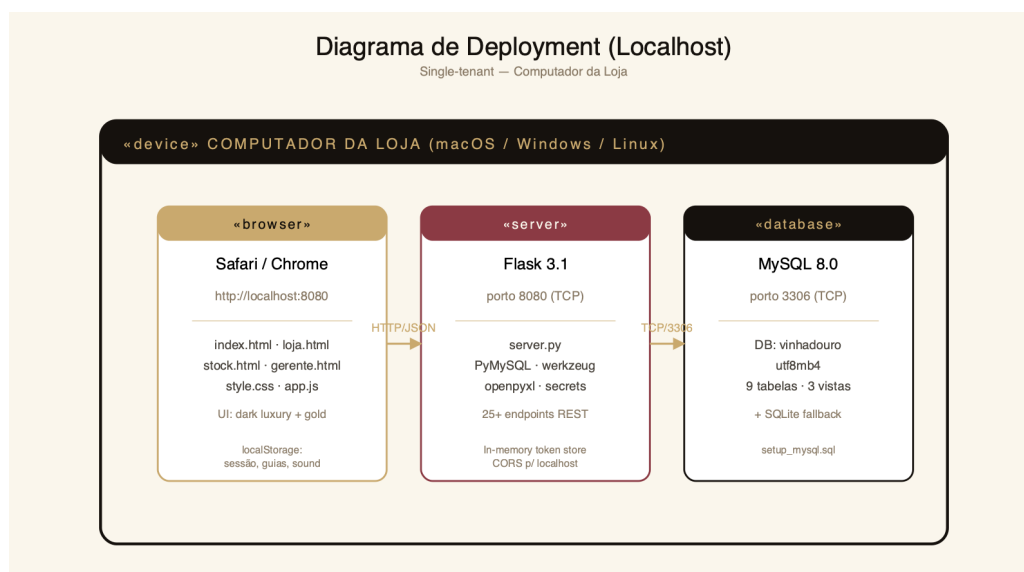


Figura 4.3. Diagrama de arquitetura física e *deployment* em ambiente `localhost`.

5.1 Objetivos do Plano de Testes

O plano de testes do *the 100's* tem como objetivos:

- Verificar que cada *endpoint* da API REST responde com os códigos HTTP corretos e o *payload* esperado.
- Validar as regras de negócio críticas (débito atômico de *stock*, geração única de códigos de venda, integridade na devolução).
- Garantir que o controle de acessos (*tokens Bearer* + papéis) funciona corretamente em todos os cenários.
- Confirmar os fluxos completos *end-to-end* (E2E) da perspectiva do utilizador.

5.2 Descrição dos Níveis de Integridade

Foram definidos três níveis de integridade:

- **Nível 1 — Crítico:** funcionalidades sem as quais o sistema não é utilizável (autenticação, cálculo do total, débito de *stock*, persistência da venda). Cobertura obrigatória.
- **Nível 2 — Importante:** funcionalidades que afetam a qualidade da experiência (relatórios, *dashboard*, *exports* Excel). Cobertura desejável.
- **Nível 3 — Acessório:** animações, transições visuais, mensagens informativas. Cobertura opcional.

5.3 Diagramas de Sequência de Integração

Os diagramas de sequência seguintes representam o fluxo de mensagens trocado entre as camadas aplicacionais durante a execução dos testes e validações E2E.

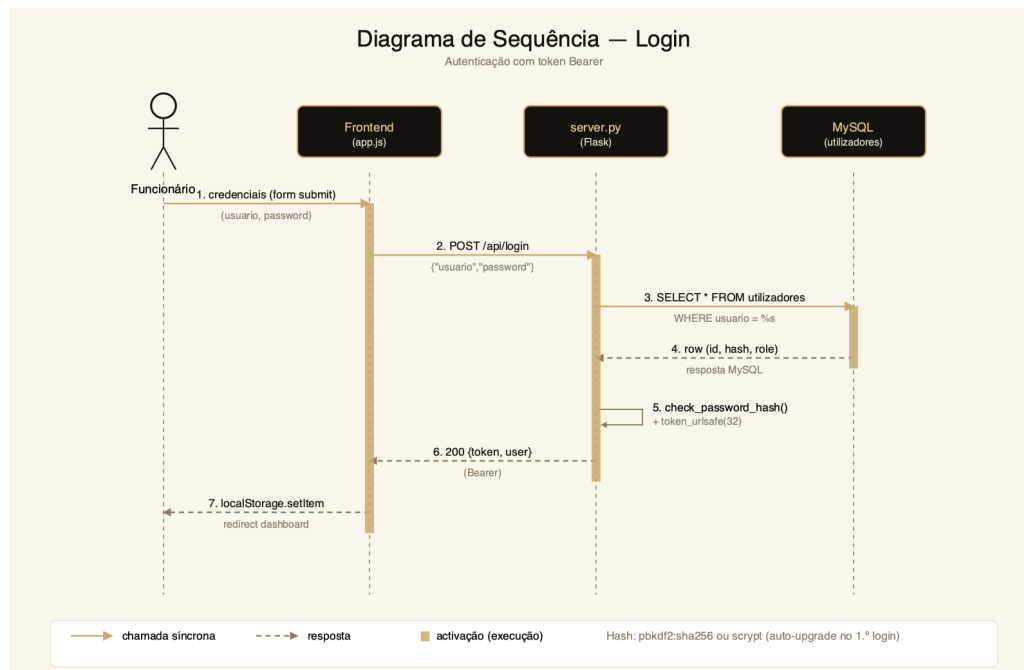


Figura 5.1. Diagrama de Sequência DS01: Processo de *login* e emissão de *token*.

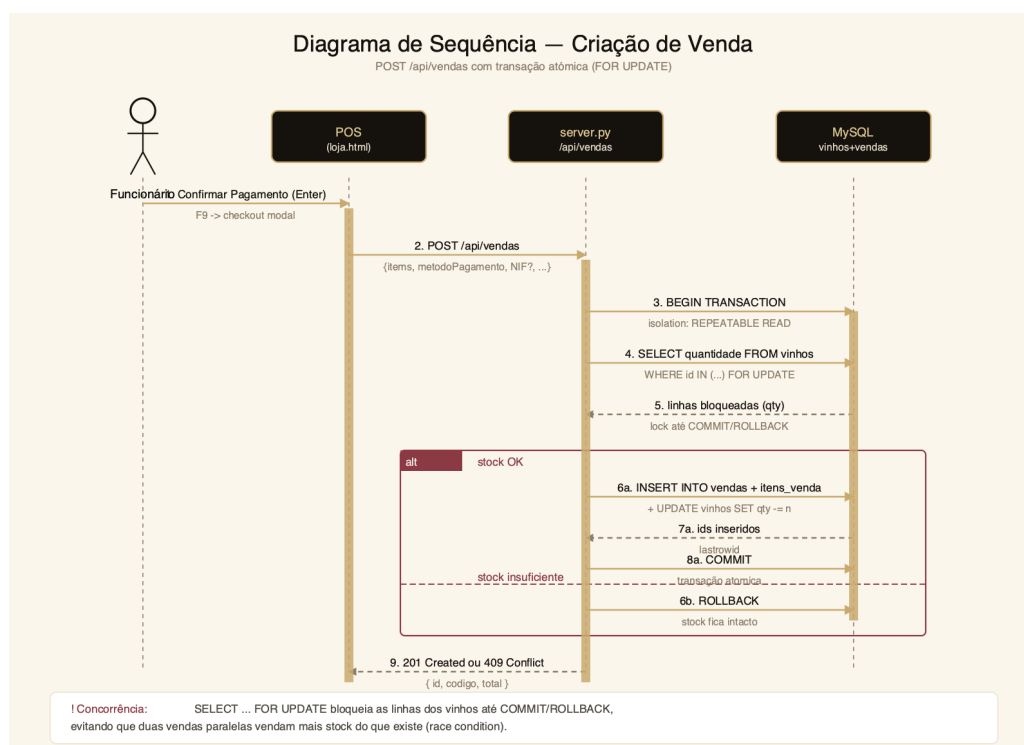


Figura 5.2. Diagrama de Sequência DS02: Processo de *checkout* com débito atômico de *stock*.

5.4 Tipos de Testes Realizados

5.4.1 Testes Unitários

Foram realizados testes unitários a funções auxiliares do *backend* (cálculo de subtotais, geração de códigos de venda, formatação de respostas JSON) e do *frontend* (`escHtml` para *escape* de HTML, `fmt` para formatação de valores monetários).

5.4.2 Testes de Integração da API

Os *endpoints* foram testados com o *Postman* / *Insomnia*, verificando:

- `POST /api/login` — retorna *token* válido com credenciais corretas; 401 com credenciais inválidas.
- `GET /api/vinhos` — retorna lista com *token* válido; 401 sem *token*.
- `POST /api/vendas` — cria venda com sucesso; 409 quando o *stock* é insuficiente.
- `POST /api/devolucoes` — processa devolução; 404 para venda inexistente.
- `GET /api/export/relatorio.xlsx` — gera ficheiro Excel com 4 folhas.

5.4.3 Testes E2E (Fluxos Completos)

Foram executados manualmente os seguintes fluxos completos:

1. **Fluxo de Loja:** *login* → catálogo → adicionar ao carrinho → *checkout* → emissão de fatura QR → envio por Email/SMS/WhatsApp.
2. **Fluxo de Armazém:** *login* como armazenista → consulta de *stock* baixo → registo de entrada → atualização visual da cave.
3. **Fluxo de Gestão:** *login* como gerente → *dashboard* → relatório mensal → *export* Excel → análise.
4. **Fluxo de Devolução:** pesquisa de venda → seleção de itens → confirmar → verificar reposição de *stock* e registo de movimento.

5.5 Geração de Dados para Testes

Os dados de teste foram gerados de três formas:

- **Automaticamente** via *seed* do `setup_mysql.sql` (22 vinhos, 8 vendas, 12 movimentos).
- **Manualmente** via *Postman*, enviando *payloads* JSON específicos para testar cenários de erro (*stock* insuficiente, *token* expirado).
- **Programaticamente** via pequeno gerador em Python que produz centenas de vendas pseudoaleatórias para testes de carga dos relatórios.

5.6 Critérios de Aceitação por Fase

Fase	Critério de Aceitação
Backend	<i>Login</i> gera <i>token</i> válido; CRUDs retornam códigos HTTP corretos; débito atômico de <i>stock</i> retorna 409 quando aplicável.
Integração	UI autentica e carrega dados reais da API; erros apresentados ao utilizador de forma clara; fluxos E2E executáveis sem passos manuais externos.
Testes	Todos os testes unitários passam; fluxos E2E manuais concluídos sem falhas críticas; <i>fallback</i> para SQLite validado.

Tabela 5.1. Critérios de aceitação por fase de desenvolvimento.

6 Formação

6.1 Métodos de Formação

A formação dos utilizadores finais do *the 100's* assenta nos seguintes métodos:

- **Manual em PDF:** documento estruturado por perfil (Gerente, Funcionário, Armazenista, Admin), descrevendo cada funcionalidade com capturas de ecrã anotadas. Disponível como `Manual_Codigo_the100s.docx` no repositório.
- **Ajuda online integrada:** *tooltips* contextuais e mensagens informativas na interface web.
- **Vídeos curtos:** demonstração geral (3–5 min), fluxo POS (2–3 min) e *dashboard* de gestão (3–4 min).
- **Sessão presencial de *onboarding*:** sessão de formação prática realizada no ato de entrega, com demonstração dos fluxos principais.

6.2 Recursos para Cada Método

Método	Recursos Necessários	Responsável
Manual PDF	Word/Pages, capturas de ecrã anotadas	Pedro G.
Ajuda integrada	HTML/CSS/JS, ícones de informação	Kollan I.
Vídeos	Captura de ecrã (QuickTime/OBS), edição	Pedro G.
Sessão presencial	Computador com o sistema instalado, projetor	Equipa

Tabela 6.1. Recursos necessários para cada método de formação.

6.3 Bases de Dados da Formação

Para as sessões de formação e demonstração é utilizada uma cópia da base de dados `vinhadouro_demo` com dados fictícios, idêntica à de produção. Permite que cada formando experimente livremente as operações (criar vendas, devolver, alterar *stock*) sem qualquer risco. A base de dados é reposta automaticamente ao fim de cada sessão.

6.4 Duração da Formação

- **Funcionário de loja:** 30 minutos (POS + emissão de fatura).
- **Armazenista:** 45 minutos (*stock* + caves + devoluções).

- **Gerente:** 60 minutos (*dashboard*, relatórios, gestão de equipa).
- **Administrador / IT:** 90 minutos (sessão técnica, instalação, *backups*).

6.5 Participantes, Responsabilidades e Avaliação

- **Participantes:** grupos reduzidos de 3 a 5 pessoas por sessão, maximizando a interação.
- **Responsável pela formação:** Grupo 13 durante o período de garantia; depois, o gerente da loja.
- **Avaliação:** (i) exercício prático guiado durante a sessão; (ii) questionário curto de satisfação; (iii) verificação informal duas semanas depois sobre uso efetivo.

7.1 Segurança no Desenvolvimento

As práticas de segurança foram aplicadas desde o início, seguindo o princípio *defesa em profundidade*:

- **Autenticação por *tokens Bearer*:** os *tokens* têm validade de 8 horas, são gerados com `secrets.token_urlsafe` e revogados em *logout*.
- **Passwords com hash:** `pbkdf2:sha256 / scrypt` via `werkzeug.security`; *auto-upgrade* do *seed plaintext* no primeiro *login*.
- **RBAC com decoradores:** cada *endpoint* sensível tem `@requires_auth` ou `@requires_role(...)`, garantindo que nenhum recurso fica acidentalmente exposto.
- **CORS restrito:** configurado via `flask-cors` para `localhost:8080` e `127.0.0.1:8080` apenas.
- **Validação de entrada:** *payloads* JSON validados antes de processamento; IDs tipados pelo *routing* do Flask.
- **Proteção contra SQL Injection:** 100% das *queries* parametrizadas (*placeholders* `%s` no PyMySQL, `?` no SQLite).
- **Proteção contra XSS:** função `escHtml()` aplicada a todo o conteúdo da API; *toasts* construídos com `textContent`, nunca `innerHTML`.
- **Segredos fora do código:** `.env` no `.gitignore`; apenas `.env.example` versionado.

7.2 Testes de Segurança

Foram realizados os seguintes testes:

- Tentativa de acesso a *endpoints* protegidos sem *token* (esperado: 401 Unauthorized).
- Tentativa de acesso com *token* de FUNCIONARIO a recursos de GERENTE (esperado: 403 Forbidden).
- Tentativa de acesso com *token* expirado (esperado: 401 Unauthorized).
- Tentativa de *SQL injection* em campos de pesquisa — todas devolveram texto inerte sem alteração da *query*.
- Tentativa de execução de *scripts* via campos de nota — todos foram exibidos como texto sem execução.
- Verificação de que *passwords* nunca aparecem em respostas da API.

7.3 Segurança no Alojamento da Base de Dados

7.3.1 Segurança contra Acessos Não Autorizados (*Security*)

- MySQL configurado para aceitar apenas conexões locais (`bind-address = 127.0.0.1`).
- Utilizador da aplicação com privilégios mínimos (`SELECT`, `INSERT`, `UPDATE`, `DELETE`; sem `GRANT`, sem `CREATE USER`).
- *Password* do MySQL via variável de ambiente, nunca em código.
- Em produção, recomenda-se TLS na ligação cliente-MySQL e *firewall* a bloquear a porta 3306 a partir do exterior.

7.3.2 Integridade dos Dados (*Safety*)

- *Constraints* de chave primária e estrangeira em todas as tabelas.
- Tipos estritos (`DECIMAL` para preços, `INT` para quantidades, `ENUM` para estados).
- Transações SQL com `ROLLBACK` explícito em caso de erro; `COMMIT` apenas no sucesso.
- *Charset* `utf8mb4` evita corrupção de caracteres acentuados.

7.4 Realização de Backups

A estratégia de *backups* assenta na regra 3-2-1 (três cópias, em dois suportes, com uma fora-do-local):

- **Backup da base de dados:** `mysqldump` diário automático para `/backups/db/`, com retenção de 30 dias.
- **Backup semanal completo:** sincronizado para disco externo na loja e para *bucket cloud* (opcional).
- **Backup da aplicação:** Git — cada *commit* é um ponto de restauro.
- **Tempo médio de recuperação:** inferior a 15 minutos, documentado em `COMO_CORRER.md`.

8

Gestão de Risco

8.1 Impacto do Risco

Os riscos foram classificados de acordo com dois eixos: probabilidade (Baixa / Média / Alta) e impacto (Baixo / Médio / Alto / Crítico). A revisão dos riscos é feita no final de cada *sprint*.

8.2 Identificação e Análise dos Riscos

ID	Descrição do Risco	Categoria	Prob.	Impacto
R1	Atraso por sobreposição com outras UC	Processo	Alta	Alto
R2	Falha do MySQL em demonstração ao vivo	Técnico	Média	Médio
R3	Dificuldades de instalação em Windows	Técnico	Média	Médio
R4	Indisponibilidade de elemento da equipa	Pessoas	Média	Alto
R5	Perda de dados por <i>bug</i> no <i>checkout</i>	Técnico	Baixa	Alto
R6	Inconsistência de <i>stock</i> após devolução	Técnico	Baixa	Médio
R7	Conflitos de <i>merge</i> no Git	Processo	Alta	Baixo
R8	Violação accidental de RGPD nos dados de teste	Processo	Baixa	Médio

Tabela 8.1. Matriz de identificação e análise de riscos do projeto.

8.3 Plano de Resposta aos Riscos Identificados

ID	Risco	Estratégia de Mitigação
R1	Atraso por sobreposição	<i>Sprints</i> curtos (2 semanas), revisão semanal do <i>backlog</i> , alocação de <i>slack</i> .
R2	Falha do MySQL	<i>Fallback</i> automático para SQLite implementado e testado antes de cada demonstração.
R3	Instalação Windows	WINDOWS.md dedicado com <i>troubleshooting</i> ; testes regulares em máquina Windows.
R4	Indisponibilidade	Documentação cruzada de cada módulo; <i>pair programming</i> nos pontos críticos.
R5	Perda de dados	Transações SQL com ROLLBACK em todas as operações de escrita; testes específicos.
R6	Inconsistência <i>stock</i>	Testes de devolução; monitorização nos <i>logs</i> de produção.
R7	Conflitos Git	<i>Branches</i> por <i>feature</i> ; <i>merges</i> frequentes para <i>main</i> ; revisão obrigatória.
R8	RGPD	Dados de teste exclusivamente fictícios; auditoria periódica do <i>seed</i> .

Tabela 8.2. Estratégias de mitigação e resposta aos riscos identificados.

9.1 Modelo de Manutenção Adotado

Para o *the 100's* adopta-se um modelo baseado em três categorias clássicas (Lehman):

- **Corretiva:** correção de defeitos reportados após a entrega.
- **Adaptativa:** acompanhar mudanças do ambiente (nova versão do MySQL, alteração de *browser*).
- **Perfectiva:** melhoria de funcionalidades existentes a pedido do utilizador.

9.2 Apoio ao Utilizador Após a Entrega

- **Período de garantia:** 3 meses, com correção gratuita de defeitos críticos.
- **Canal de suporte:** *email* do grupo ou abertura de *issue* no GitHub; SLA de 48 horas em dias úteis.
- **Documentação:** sempre disponível e atualizada após cada *release*.

9.3 Periodicidade da Manutenção

Tipo de Manutenção	Periodicidade	Observações
Corretiva (defeitos críticos)	Resposta imediata	SLA: correção até 48h
Corretiva (defeitos menores)	Mensal	Incluída na <i>release</i> mensal
Adaptativa	Trimestral	Acompanha versões de Python, Flask, MySQL
Perfectiva	Sob procura	Avaliada caso a caso
Atualização de segurança	Sempre que necessário	Vulnerabilidades conhecidas (CVE)

Tabela 9.1. Periodicidade da manutenção por tipo.

9.4 Ações a Realizar na Manutenção

- Análise de *logs* do servidor para identificar erros recorrentes.
- Atualização de dependências em `requirements.txt` e validação em testes.
- Revisão de *queries* com tempo elevado, identificadas via *slow query log*.

- Auditoria periódica das *passwords* (atualização anual ou após incidente).
- Teste de restauro de *backup* (uma vez por trimestre).
- Atualização do conteúdo das páginas de ajuda e do manual.

9.5 Modo de Realizar a Manutenção

As intervenções seguem um fluxo controlado por Git: cada alteração é desenvolvida numa *branch* específica (*fix/...*, *feat/...*, *chore/...*), submetida via *Pull Request*, revista por outro elemento da equipa e fundida em *main* apenas após validação dos testes. Quando exigem acesso ao ambiente do cliente, são agendadas fora do horário de operação.

9.6 Responsáveis pela Manutenção

- **Eduardo Saavedra Lourenço** — *Backend* (*server.py*, autenticação) e Base de Dados.
- **Kollan Andre Gafuro Intacua** — *Frontend* (HTML, CSS, JavaScript) e POS.
- **Pedro Gabriel Matias Gomes** — Arquitetura, *dashboard*, QA e coordenação (PM).

10 Conclusões

10.1 Reflexões Finais sobre a Condução do Projeto

O desenvolvimento do *the 100's* permitiu à equipa aplicar, num contexto prático e completo, os conceitos de Engenharia de *Software* abordados na unidade curricular: planeamento estruturado em *sprints*, desenvolvimento incremental por módulos, arquitetura em camadas, controlo de versões colaborativo e documentação técnica.

A separação clara entre *frontend* e *backend*, aliada à segurança baseada em *tokens Bearer* e RBAC, produziu um sistema modular e extensível. A escolha de uma *stack* ligeira (Flask, MySQL, JavaScript *vanilla*) revelou-se acertada para o prazo do projeto: garantiu rapidez de desenvolvimento, demonstrabilidade local sem dependências de *cloud* e uma curva de aprendizagem reduzida, libertando esforço para o cuidado com a qualidade do código, da interface e da documentação.

O trabalho em equipa revelou-se fundamental: a divisão de responsabilidades por domínio técnico, com colaboração transversal, permitiu entregar um produto funcional dentro do prazo estabelecido. A identidade *Bottled Memories* conferiu ao *the 100's* uma personalidade distintiva, raramente presente em projetos académicos, e que valoriza o trabalho perante uma audiência mais ampla.

10.2 Áreas a Melhorar

- **Cobertura de testes automatizados:** a cobertura atual deve evoluir para uma *suite* com `pytest` e `jest`, integrada em CI/CD via GitHub Actions.
- **Internacionalização:** o sistema está hoje apenas em português; preparar para inglês alargaria o público potencial.
- **Acessibilidade (*a11y*):** aumentar o contraste de alguns elementos da paleta *gold + ink* e completar a navegação por teclado.
- **Certificação fiscal (AT):** o sistema já está preparado arquiteturalmente (ATCUD, QR, *hash*), mas falta percorrer o processo formal de certificação.
- **Modularização do `server.py`:** o ficheiro tem mais de 1700 linhas e beneficiaria de reorganização em *blueprints* Flask.
- **Observabilidade:** adicionar métricas estruturadas (Prometheus) e *logs* JSON para facilitar a operação em produção.

10.3 Perspetivas de Trabalho Futuro

O *roadmap* de evolução prevê:

- **Módulo de Faturação fiscal:** certificação AT e emissão de faturas com valor fiscal.
- **Pagamentos reais:** integração com MB WAY via API oficial e terminais de pagamento físicos.
- **Cobertura automática de código:** integração de `pytest` e `jest` em *pipeline* CI/CD.
- **App nativa:** versão móvel para os vendedores no espaço da loja.
- **Analítica avançada:** previsões de vendas e recomendações personalizadas via *machine learning*.
- **Arquitetura multi-loja:** suporte a múltiplas lojas sob a mesma plataforma, com sincronização *cloud*.

Código-Fonte do Projeto

O código-fonte completo do *the 100's*, incluindo *backend*, *frontend*, *scripts* de base de dados e documentação técnica, está disponível em:

<https://github.com/PedroGGomess/Vinha-D-Ouro>

Referências

- [1] Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9.^a ed.). McGraw-Hill.
- [2] Sommerville, I. (2016). *Software Engineering* (10.^a ed.). Pearson.
- [3] Strecht, P. (2025). *ESW_P12 — O Relatório de Projeto de Software*. Universidade Lusófona, Centro Universitário do Porto.
- [4] Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of Database Systems* (7.^a ed.). Pearson.
- [5] OWASP Foundation (2021). *OWASP Top 10:2021 — The Ten Most Critical Web Application Security Risks*. <https://owasp.org/Top10/>
- [6] Comissão Europeia (2016). *Regulamento (UE) 2016/679 — Regulamento Geral sobre a Proteção de Dados (RGPD)*. <https://eur-lex.europa.eu/eli/reg/2016/679/oj>
- [7] Pallets Projects. *Flask Documentation (v3.x)*. <https://flask.palletsprojects.com/>
- [8] Oracle Corporation. *MySQL 8.0 Reference Manual*. <https://dev.mysql.com/doc/refman/8.0/en/>
- [9] Python Software Foundation. *The Python Language Reference (3.10+)*. <https://docs.python.org/3/>
- [10] Mozilla Developer Network. *Web Docs — HTML, CSS, JavaScript*. <https://developer.mozilla.org/>
- [11] openpyxl. *A Python library to read/write Excel 2010 xlsx/xlsm files*. <https://openpyxl.readthedocs.io/>
- [12] Gomes, P. G. M., Lourenço, E. S., & Intacua, K. A. G. (2026). *Repositório do projeto the 100's — Vinha-D-Ouro*. <https://github.com/PedroGGomess/Vinha-D-Ouro>